HypArr

Computations with hyperplane arrangements

0.26.04

28 April 2026

Paul Mücksch

Paul Mücksch
Email: paul.muecksch+uni@gmail.com
Homepage: https://paulmuecksch.github.io
Address: Leibniz Universität Hannover
Institut für Algebra, Zahlentheorie und Diskrete Mathematik
Welfengarten 1
30167 Hannover, Germany

Contents

1 Introduction **4**

2 Hyperplane arrangements **5**
2.1 Construction of arrangements of hyperplanes 5
2.2 Attributes of arrangements 5
2.3 Properties of arrangements 7
2.4 Attributes of geometric lattices 8
2.5 Properties of geometric lattices 9
2.6 Operations 10

3 Special arrangements **12**
3.1 Global methods 12

4 Oriented Matroids **14**
4.1 Construction of oriented matroids 14
4.2 Attributes 14
4.3 Properties 18
4.4 Operations 19
4.5 Global methods 20

5 Free arrangements **22**
5.1 Module of logarithmic vector fields aka derivation modules of arrangements 22
5.2 Freeness properties 24

6 Further properties of arrangements, geometric lattices, and oriented matroids **26**
6.1 Supersolvable arrangements 26
6.2 Formal arrangements 28
6.3 Simplicial arrangements 29
6.4 Falk's weight test 29
6.5 Factored arrangements 30

7 Milnor fibers **33**
7.1 Complexes 33

8 Tope posets **35**
8.1 Operation 35
8.2 Attributes 35

9 Visualization of real 3-arrangements **37**
9.1 Global methods 37

10 Realization spaces of geometric lattices **43**
10.1 Construction 43
10.2 Attributes 43
10.3 Properties 45
10.4 Further (auxillary) Operations 45

References **46**

Index **47**

Chapter 1

Introduction

HYPARR is a GAP 4 package for computations with hyperplane arrangements (see [OT92]), oriented matroids (see [BLVS+99]), their combinatorial and topological invariants, in particular geometric lattices, face posets, Milnor fibers and complements.

Chapter 2

Hyperplane arrangements

Let $\mathbb{K}$ be a field. A hyperplane arrangment is a finite set $\mathscr{A} = \{H_1, \ldots, H_n\}$ of hyperplanes in a finite dimensional $\mathbb{K}$-vector space V.

2.1 Construction of arrangements of hyperplanes

2.1.1 HyperplaneArrangement (for IsList)

▷ `HyperplaneArrangement(R)` (operation)

Returns: A hyperplane arrangement

Constructs a hyperplane arrangement from a list `R` of vectors representing defining linear forms of hyperplanes.

Each vector $[a_1, \ldots, a_\ell]$ in `R` corresponds to the hyperplane

$$a_1 x_1 + \cdots + a_\ell x_\ell = 0.$$

The vectors must lie in the same vector space.

Linearly dependent defining forms are removed automatically.

Example

```
gap> A := HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1]]);
<HyperplaneArrangement: 3 hyperplanes in 3-space>
gap> Roots(A);
[ [ 1, 0, 0 ], [ 0, 1, 0 ], [ 0, 0, 1 ] ]
```

2.2 Attributes of arrangements

2.2.1 Dimension (for IsHyperplaneArrangement)

▷ `Dimension(A)` (attribute)

Returns: A non-negative integer.

Returns the dimension of the ambient vector space in which the hyperplane arrangement `A` is defined.

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0]]); Dimension(A);
<HyperplaneArrangement: 2 hyperplanes in 3-space>
3
```

2.2.2 Roots (for IsHyperplaneArrangement)

▷ `Roots(A)` (attribute)

Returns: A list of vectors.

Returns the list of vectors giving defining linear forms (also called *roots*) of the hyperplanes in the arrangement `A`.

Example
```
gap> A:=AGpql(3,3,2); Roots(A);
<HyperplaneArrangement: 3 hyperplanes in 2-space>
[ [ 1, -E(3) ], [ 1, -E(3)^2 ], [ 1, -1 ] ]
```

2.2.3 HArrDefField (for IsHyperplaneArrangement)

▷ `HArrDefField(A)` (attribute)

Returns: A field

Returns the field of definition of `A`. For further compuations, in characterisic 0 only cyclotomic fields are considered.

Example
```
gap> A:=HyperplaneArrangement(Z(3)^0*[[1,0],[0,1],[1,1]]); HArrDefField(A);
<HyperplaneArrangement: 3 hyperplanes in 2-space>
GF(3)
gap> A:=AGpql(5,5,2); HArrDefField(A);
<HyperplaneArrangement: 5 hyperplanes in 2-space>
CF(5)
```

2.2.4 IntersectionLattice (for IsHyperplaneArrangement)

▷ `IntersectionLattice(A)` (attribute)

Returns: An object of type `IsGeomLattice`.

Computes the intersection lattice of the hyperplane arrangement `A`.

The ground set `L` (see `LGroundSet` (2.4.1)) of the lattice is represented level-by-level. The entry `L[k]` contains all intersections of rank k.

Each lattice element is stored as a list of indices referring to hyperplanes in `Roots(A)`.

For example, the list `[1,3]` represents the intersection of the first and third hyperplane of the arrangement.

Example
```
gap> A := HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,0],[1,1,1]]);
<HyperplaneArrangement: 3 hyperplanes in 3-space>
gap> L:=IntersectionLattice(A); LGroundSet(L);
<Geometric lattice: 3 atoms, rank 3>
[ [ [ 1 ], [ 2 ], [ 3 ] ],
  [ [ 1, 2 ], [ 1, 3 ], [ 2, 3 ] ],
  [ [ 1, 2, 3 ] ] ]
```

2.2.5 CharPoly (for IsHyperplaneArrangement)

▷ `CharPoly(A)` (attribute)

Returns: a univariate polynomial with integral coefficients.

Returns $\chi_{\mathscr{A}}$ the *characterisitc polynomial* of the arrangement `A`.

Example
```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> CharPoly(A);
t^3-6*t^2+11*t-6
```

2.2.6 ExpArr (for IsHyperplaneArrangement)

▷ `ExpArr(A)` (attribute)

Returns: a list of integers, or fail

Returns *exponents* of the arrangement $\mathscr{A}$ if the characteristic polynomial factors over the integers.

Example
```
gap> A:=HyperplaneArrangement([[1,0],[0,1],[1,1]]);
<HyperplaneArrangement: 3 hyperplanes in 2-space>
gap> ExpArr(A);
[ 1, 2 ]
```

2.2.7 MSetInvL (for IsHyperplaneArrangement)

▷ `MSetInvL(A)` (attribute)

Returns: An object of type IsGeomLattice.

Computes multiset invariants of the intersection lattice (see `IntersectionLattice` (2.2.4)) of the hyperplane arrangement `A`.

For each level of the lattice, the function records how many intersections occur with a given number of defining hyperplanes.

Example
```
gap> A:=HyperplaneArrangement([[1,0,0,],[0,1,0],[0,0,1],[1,1,1]]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
gap> MSetInvL(A);
[ [ [ 1, 4 ] ], [ [ 2, 6 ] ], [ [ 4, 1 ] ] ]
```

2.3 Properties of arrangements

2.3.1 IsReal (for IsHyperplaneArrangement)

▷ `IsReal(A)` (property)

Returns: true or false.

Determines, if the hyperplane arrangement is defined over the reals, i.e., if the entries of the defining linear forms are real.

2.3.2 HArrIsIrreducible (for IsHyperplaneArrangement)

▷ `HArrIsIrreducible(A)` (property)

Returns: true or false.

Determines, if the hyperplane arrangement is irreducible.

2.3.3 HArrIsGeneric (for IsHyperplaneArrangement)

▷ `HArrIsGeneric(A)` (property)

Returns: true or false.

Determines, if the hyperplane arrangement is generic. See also `LIsGeneric` (2.5.3).

Example

```
gap> A := HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> HArrIsGeneric(A);
false
gap> A := HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
gap> HArrIsGeneric(A);
true
```

2.4 Attributes of geometric lattices

2.4.1 LGroundSet (for IsGeomLattice)

▷ `LGroundSet(L)` (attribute)

Returns: list of lists

Returns the ground set of the geometric lattice `L`.

2.4.2 LAtoms (for IsGeomLattice)

▷ `LAtoms(L)` (attribute)

Returns: list

Returns the set of atoms of `L`.

2.4.3 LRank (for IsGeomLattice)

▷ `LRank(L)` (attribute)

Returns: a non-negative integer

Returns the rank of `L`.

2.4.4 LkFlats (for IsGeomLattice)

▷ `LkFlats(L)` (attribute)

Returns: a function

Returns a function extracting the flats of rank k in `L`.

2.4.5 LRankFunction (for IsGeomLattice)

▷ `LRankFunction(L)` (attribute)

Returns: A function

Returns the rank function of the geometric lattice `L`.

2.4.6 LGraph (for IsGeomLattice)

▷ `LGraph(L)` (attribute)

Returns: a graph

Constructs the directed graph of the Hasse diagram of L.

2.4.7 LAutGroup (for IsGeomLattice)

▷ `LAutGroup(L)` (attribute)

Returns: a group

Computes the autmorphism group of L as a subgroup of `Sym(LAtoms(L))`.

2.4.8 LMoebius (for IsGeomLattice)

▷ `LMoebius(L)` (attribute)

Returns: function

Returns the Moebius function of `L`.

2.4.9 LCharPoly (for IsGeomLattice)

▷ `LCharPoly(L)` (attribute)

Returns: function

Computes the characteristic polynomial of `L`.

2.4.10 LBases (for IsGeomLattice)

▷ `LBases(L)` (attribute)

Returns: list of bases

Determines the subsets of `LAtoms(L)` which are bases.

Example

```
```

2.5 Properties of geometric lattices

2.5.1 LIsIrreducible (for IsGeomLattice)

▷ `LIsIrreducible(L)` (property)

Returns: true or false.

Determines, if L is irreducible.

2.5.2 LIsBoolean (for IsGeomLattice)

▷ `LIsBoolean(L)` (property)

Returns: true or false.

Determines, if L is a boolean lattice.

Example
```
gap> L:=IntersectionLattice(HyperplaneArrangement([[1,0],[1,1]]));
<Geometric lattice: 2 atoms, rank 2>
gap> LIsBoolean(L);
true
gap> L:=IntersectionLattice(HyperplaneArrangement([[1,0],[0,1],[1,1]]));
<Geometric lattice: 3 atoms, rank 2>
gap> LIsBoolean(L);
false
```

2.5.3 LIsGeneric (for IsGeomLattice)

▷ `LIsGeneric(L)` (property)

Returns: true or false.

Determines, if L is generic, i.e. if L is irreducible and all proper intervals are boolean.

Example
```
gap> A := HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
gap> L:=IntersectionLattice(A);
<Geometric lattice: 4 atoms, rank 3>
gap> LIsGeneric(L);
true
gap> A := HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> L:=IntersectionLattice(A);
<Geometric lattice: 5 atoms, rank 3>
gap> LIsGeneric(L);
false
```

2.6 Operations

2.6.1 LIsIsomorphic (for IsGeomLattice,IsGeomLattice)

▷ `LIsIsomorphic(LA, LB)` (operation)

Returns: true or false

Determines if the geometric lattices *LA* and *LB* are isomorphic.

Example
```
gap> A:=HyperplaneArrangement([[1,0],[0,1],[1,1]]); B:=HyperplaneArrangement([[1,0],[1,1],[1,2]])
<HyperplaneArrangement: 3 hyperplanes in 2-space>
<HyperplaneArrangement: 3 hyperplanes in 2-space>
gap> LA:=IntersectionLattice(A); LB:=IntersectionLattice(B);
<Geometric lattice: 3 atoms, rank 2>
<Geometric lattice: 3 atoms, rank 2>
gap> LIsIsomorphic(LA,LB);
true
```

2.6.2 IsLEquiv (for IsHyperplaneArrangement,IsHyperplaneArrangement)

▷ `IsLEquiv(A, B)` (operation)

Returns: true or false

Determines if the arrangements *A* and *B* have isomorphic intersection lattices.

Example
```
gap> A:=HyperplaneArrangement(([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[0,1,1],[1,1,3]]);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> B:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> IsLEquiv(A,B);
false
gap> A:=HyperplaneArrangement(([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[0,1,1],[1,1,1]]);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> IsLEquiv(A,B);
true
```

2.6.3 Essentialization (for IsHyperplaneArrangement)

▷ `Essentialization(A)` (operation)

Returns: A hyperplane arrangement.

Computes the *essentialization* of the hyperplane arrangement *A*. An arrangement is called essential if the intersection of all its hyperplanes is the origin. If this is not the case, the function restricts the arrangement to a complementary subspace so that the resulting arrangement becomes essential.

Example
```
gap> A:=HyperplaneArrangement(([[1,0,0],[0,1,0],[1,1,0]]);
<HyperplaneArrangement: 3 hyperplanes in 3-space>
gap> B:=Essentialization(A); Roots(B);
<HyperplaneArrangement: 3 hyperplanes in 2-space>
[ [ 1, 0 ], [ 0, 1 ], [ 1, 1 ] ]
```

2.6.4 HArrRestriction (for IsHyperplaneArrangement, IsInt)

▷ `HArrRestriction(A, i)` (operation)

Returns: A hyperplane arrangement.

Computes the *restriction* of the arrangement *A* to the *i*-th hyperplane of `Roots(A)`.

2.6.5 HArrRestriction (for IsHyperplaneArrangement, IsList)

▷ `HArrRestriction(A, S)` (operation)

Returns: A hyperplane arrangement.

Computes the *restriction* of the arrangement *A* to the subspace orthogonal to the vectors in `S`.

Chapter 3

Special arrangements

The following describes functions to construct some special arrangements.

3.1 Global methods

3.1.1 AGpql

▷ `AGpql(p, q, l)` (function)

Returns: A hyperplane arrangement.

Constructs the reflection arrangement associated to the monomial complex reflection group $G(p,q,l)$. The hyperplanes are defined by equations of the form

$$x_i = \zeta^k x_j$$

where ζ is a primitive p-th root of unity.

Example

```
gap> A := AGpql(2,1,3);
 <HyperplaneArrangement: 9 hyperplanes in 3-space>
gap> Roots(A);
[ [ 1, 0, 0 ], [ 0, 1, 0 ], [ 0, 0, 1 ],
  [ 1, 1, 0 ], [ 1, -1, 0 ], [ 1, 0, 1 ],
  [ 1, 0, -1 ], [ 0, 1, 1 ], [ 0, 1, -1 ] ]
```

3.1.2 SsS3

▷ `SsS3(n)` (function)

Returns: A hyperplane arrangement.

Generates the irreducible supersolvable simplicial arrangement of rank 3 with n hyperplanes. n must either be even oder congruent 1 mod 4. In any case, $n \geq 6$.

Example

```
gap> A:=SsS3(9); Roots(A);
<HyperplaneArrangement: 9 hyperplanes in 3-space>
[ [ 0, 0, 1 ], [ 0, 1, 0 ],
  [ -1/2*E(8)+1/2*E(8)^3, 1/2*E(8)-1/2*E(8)^3, 0 ],
  [ -1, 0, 0 ],
  [ -1/2*E(8)+1/2*E(8)^3, -1/2*E(8)+1/2*E(8)^3, 0 ],
```

```
  [ 1/2*E(8)-1/2*E(8)^3, 1/2*E(8)-1/2*E(8)^3, 1 ],
  [ -1/2*E(8)+1/2*E(8)^3, 1/2*E(8)-1/2*E(8)^3, 1 ],
  [ -1/2*E(8)+1/2*E(8)^3, -1/2*E(8)+1/2*E(8)^3, 1 ],
  [ 1/2*E(8)-1/2*E(8)^3, -1/2*E(8)+1/2*E(8)^3, 1 ] ]
```

3.1.3 ConnectedSubgraphArr (for IsList)

▷ `ConnectedSubgraphArr(Es)` (operation)

Returns: A hyperplane arrangement.

Generates the connected subgraph arrangement (see [CK22]) of the graph with edges `Es`.

Let $G = (N, E)$ be an undirected graph with vertex set $N = [n]$ and edge set E. For $I \subseteq N$, let $G[I]$ be the induced subgraph of G on the set of vertices I. For $I \subseteq N$, define the hyperplane

$$H_I := \ker \sum_{i \in I} x_i.$$

The connected subgraph arrangement consists of hyperplanes $\{H_I \mid \varnothing \neq I \subseteq N \text{ if } G[I] \text{ is connected }\}$.

Example

```
gap> A:=ConnectedSubgraphArr([[1,2],[1,3],[2,3]]); Roots(A);
<HyperplaneArrangement: 7 hyperplanes in 3-space>
[ [ 1, 1, 1 ], [ 0, 1, 1 ], [ 0, 0, 1 ], [ 0, 1, 0 ],
  [ 1, 0, 1 ], [ 1, 0, 0 ], [ 1, 1, 0 ] ]
```

Chapter 4

Oriented Matroids

Oriented matroids are abstractions of real hyperplane arrangements. The following describes functions to handle oriented matroids and compute associated cell complexes and invariants. See [BLVS+99].

4.1 Construction of oriented matroids

4.1.1 OrientedMatroid (for IsList)

▷ `OrientedMatroid(R, or, A, or[, CCs, c], or[, n, r, ChiroCoreList])` (operation)

Returns: An oriented matroid OM.

Constructs am oriented matroid from

- a list *R* of vectors representing defining linear forms of real hyperplanes,
- a real hyperplane arrangement *A*,
- a list of lists *CVs* of signed covectors and a string `c="cv"`,
- intergers *n,r* giving the size of the groundset and the rank and a list *ChiroCoreList* $\subseteq \{0,1,-1\}^m (m = \binom{n}{r})$ presenting the chirotope

of the oriented matroid.

Example

```
gap> A:=AGpql(2,2,3); OM:=OrientedMatroid(Roots(A));
<OrientedMatroid: 6 elements, rank 3>
gap> A:=AGpql(2,2,3); OM:=OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OrientedMatroid([[[1,1],[-1,1],[1,-1],[-1,-1]],[[0,1],[0,-1],[1,0],[-1,0]],[[0,0]]],"cv");
<OrientedMatroid: 2 elements, rank 2>
gap> OrientedMatroid(2,3,[1,1,1]);
<OrientedMatroid: 3 elements, rank 2>
```

4.2 Attributes

4.2.1 OMRank (for IsOrientedMatroid)

▷ `OMRank(OM)` (attribute)

Returns: An integer

Returns the rank of the oriented matroid *OM*.

Example
```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OMRank(OM);
3
```

4.2.2 OMGroundSet (for IsOrientedMatroid)

▷ `OMGroundSet(OM)` (attribute)

Returns: A list

Returns the ground set of the oriented matroid OM.

Example
```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OMGroundSet(OM);
[ 1, 2, 3, 4, 5, 6 ]
```

4.2.3 OMLForms (for IsOrientedMatroid)

▷ `OMLForms(OM)` (attribute)

Returns: A list of linear forms or `fail`

Returns the list of linear forms associated with the oriented matroid OM. If the internal component `OM!.lforms` is not bound, `fail` is returned.

Example
```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OMLForms(O);
[ [ 1, 1, 0 ], [ 1, -1, 0 ], [ 1, 0, 1 ],
  [ 1, 0, -1 ], [ 0, 1, 1 ], [ 0, 1, -1 ] ]
```

4.2.4 OMGeomLattice (for IsOrientedMatroid)

▷ `OMGeomLattice(OM)` (attribute)

Returns: An object of type `IsGeomLattice` or `fail`

Constructs the geometric lattice associated with the oriented matroid OM. If OM is linear (as determined by `OMIsLinear(OM)`), the lattice is computed as the intersection lattice of the hyperplane arrangement given by `OMLForms(OM)`, stored in the internal component `OM!.lattice`, and returned.

Example
```
gap> O := OrientedMatroid([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<OrientedMatroid: 4 elements, rank 3>
gap> L:=OMGeomLattice(O); LGroundSet(L);
<Geometric lattice: 4 atoms, rank 3>
[ [ [ 1 ], [ 2 ], [ 3 ], [ 4 ] ],
  [ [ 1, 2 ], [ 1, 3 ], [ 2, 3 ],
      [ 1, 4 ], [ 2, 4 ], [ 3, 4 ] ],
  [ [ 1, 2, 3, 4 ] ] ]
gap> O := OrientedMatroid(3,4,[1,1,1,1]);
<OrientedMatroid: 4 elements, rank 3>
gap> L:=OMGeomLattice(O); LGroundSet(L);
```

```
<Geometric lattice: 4 atoms, rank 3>
[ [ [ 1 ], [ 2 ], [ 3 ], [ 4 ] ],
  [ [ 1, 2 ], [ 1, 3 ], [ 2, 3 ],
      [ 1, 4 ], [ 2, 4 ], [ 3, 4 ] ],
  [ [ 1, 2, 3, 4 ] ] ]
```

4.2.5 OMCocircuits (for IsOrientedMatroid)

▷ `OMCocircuits(OM)` (attribute)

Returns: A list

Computes the (signed) cocircuits of the oriented matroid OM.

The result is a list of sign vectors.

Example

```
gap> O:=OrientedMatroid([[1,0],[0,1],[1,1]]); OMCocircuits(O);
<OrientedMatroid: 3 elements, rank 2>
[ [ 0, 1, 1 ], [ 0, -1, -1 ], [ 1, 0, 1 ],
  [ -1, 0, -1 ], [ -1, 1, 0 ], [ 1, -1, 0 ] ]
```

4.2.6 TopeGraph (for IsOrientedMatroid)

▷ `TopeGraph(OM)` (attribute)

Returns: A graph

Constructs the tope graph of the oriented matroid *OM*. The topes are obtained as the maximal covectors from `OMCovectors(OM)`. Two topes are connected by an edge if their separating set consists of one element.

Example

```
gap> O := OrientedMatroid([[1,0],[0,1],[1,1]]);
<OrientedMatroid: 3 elements, rank 2>
gap> G := ShallowCopy(TopeGraph(O));
gap> Vertices(G);
[ 1 .. 6 ]
gap> IsSimpleGraph(G);
true
gap> UndirectedEdges(G);
[ [ 1, 3 ], [ 1, 5 ], [ 2, 4 ], [ 2, 6 ], [ 3, 6 ], [ 4, 5 ] ]
```

4.2.7 OMCovectors (for IsOrientedMatroid)

▷ `OMCovectors(OM)` (attribute)

Returns: A list of lists

Computes all covectors of the oriented matroid OM. The result is returned as a list of lists, where the k-th inner list is a list of covectors corresponding to faces of dimension $k-1$.

Example

```
gap> O := OrientedMatroid([[1,0],[0,1],[1,1]]);
<OrientedMatroid: 3 elements, rank 2>
gap> OMCovectors(O);
[ [ [ -1, 1, -1 ], [ 1, -1, 1 ], [ -1, -1, -1 ],
    [ 1, 1, 1 ], [ -1, 1, 1 ], [ 1, -1, -1 ] ],
  [ [ 0, 1, 1 ], [ 0, -1, -1 ], [ 1, 0, 1 ],
```

```
      [ -1, 0, -1 ], [ -1, 1, 0 ], [ 1, -1, 0 ] ],
    [ [ 0, 0, 0 ] ] ]
```

4.2.8 SalvettiComplex (for IsOrientedMatroid)

▷ `SalvettiComplex(OM, or, A)` (attribute)

Returns: A `FacePoset`

Constructs the Salvetti complex [Sal87] associated with the oriented matroid OM or the real hyperplane arrangement *A*.

Example

```
gap> O := OrientedMatroid([[1,0],[0,1],[1,1]]);
<OrientedMatroid: 3 elements, rank 2>
gap> SalvettiComplex(O);
<FacePoset of dimension 2 with f-vector [ 6, 12, 6 ]>
gap> A:=HyperplaneArrangement([[1,0],[0,1],[1,1]]);
<HyperplaneArrangement: 3 hyperplanes in 2-space>
gap> SalvettiComplex(A);
<FacePoset of dimension 2 with f-vector [ 6, 12, 6 ]>
```

4.2.9 SalvettiComplex (for IsHyperplaneArrangement)

▷ `SalvettiComplex(A)` (attribute)

See `SalvettiComplex` (4.2.8).

4.2.10 FPGroundSet (for IsFacePoset)

▷ `FPGroundSet(FP)` (attribute)

Returns: A list

Returns the ground set of the face poset FP.

Example

```
gap> S := SalvettiComplex(O);
<FacePoset of dimension 2 with f-vector [ 6, 12, 6 ]>
gap> FPGroundSet(S);
[
  [
    [ [ -1, 1, -1 ], [ -1, 1, -1 ] ],
    [ [ 1, -1, 1 ], [ 1, -1, 1 ] ],
    [ [ -1, -1, -1 ], [ -1, -1, -1 ] ],
    [ [ 1, 1, 1 ], [ 1, 1, 1 ] ],
    [ [ -1, 1, 1 ], [ -1, 1, 1 ] ],
    [ [ 1, -1, -1 ], [ 1, -1, -1 ] ]
  ],
  [
    [ [ -1, 0, -1 ], [ -1, 1, -1 ] ],
    [ [ -1, 1, 0 ], [ -1, 1, -1 ] ],
    [ [ 1, 0, 1 ], [ 1, -1, 1 ] ],
    [ [ 1, -1, 0 ], [ 1, -1, 1 ] ],
    [ [ 0, -1, -1 ], [ -1, -1, -1 ] ],
    [ [ -1, 0, -1 ], [ -1, -1, -1 ] ],
```

```
      [ [ 0, 1, 1 ], [ 1, 1, 1 ] ],
      [ [ 1, 0, 1 ], [ 1, 1, 1 ] ],
      [ [ 0, 1, 1 ], [ -1, 1, 1 ] ],
      [ [ -1, 1, 0 ], [ -1, 1, 1 ] ],
      [ [ 0, -1, -1 ], [ 1, -1, -1 ] ],
      [ [ 1, -1, 0 ], [ 1, -1, -1 ] ]
    ],
    [
      [ [ 0, 0, 0 ], [ -1, 1, -1 ] ],
      [ [ 0, 0, 0 ], [ 1, -1, 1 ] ],
      [ [ 0, 0, 0 ], [ -1, -1, -1 ] ],
      [ [ 0, 0, 0 ], [ 1, 1, 1 ] ],
      [ [ 0, 0, 0 ], [ -1, 1, 1 ] ],
      [ [ 0, 0, 0 ], [ 1, -1, -1 ] ]
    ]
  ]
```

4.2.11 FPOrder (for IsFacePoset)

▷ `FPOrder(FP)` (attribute)

Returns: A function

Returns the order function of the face poset FP.

4.2.12 OMChirotope (for IsOrientedMatroid)

▷ `OMChirotope(OM)` (attribute)

Returns: A function

Returns the chirotope given as a function from $\binom{E}{r} \to \{0, \pm 1\}$, where E is the groundset of OM and r is the rank of `OM`.

Example

```
gap> O:=OrientedMatroid(2,3,[1,1,1]); c:=OMChirotope(O);
<OrientedMatroid: 3 elements, rank 2>
function( S ) ... end
gap> List(Combinations([1..3],2),x->c(x));
[ 1, 1, 1 ]
```

4.2.13 OMRankFunction (for IsOrientedMatroid)

▷ `OMRankFunction(OM)` (attribute)

Returns: A function

Returns the rank function of the oriented matroid `OM`

4.3 Properties

4.3.1 OMIsLinear (for IsOrientedMatroid)

▷ `OMIsLinear(OM)` (property)

Returns: true or false

Determines whether the oriented matroid *OM* is linear. This is the case if the internal component `OM!.lforms` is bound.

Example
```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OMIsLinear(OM);
true
gap> O := OrientedMatroid(3,4,[1,1,1,1]);
<OrientedMatroid: 4 elements, rank 3>
gap> OMIsLinear(O);
false
```

4.3.2 HasSimpleSimplex (for IsOrientedMatroid)

▷ `HasSimpleSimplex(OM)` (property)

Returns: true or false

Returns whether the oriented matroid *OM* has a simple simplex, i.e. a simplicial Tope T such that the localizations at its vertices are boolean and there is another rank 1 covector not adjacent to T. If `Rank(OM)` ≥ 3 and this is true then its Salvetti complex can not be $K(\pi,1)$.

Example
```
gap> O:=OrientedMatroid([[1,0,0,0],[0,1,0,0],[0,0,1,0],[0,0,0,1],[1,1,1,1]]);
<OrientedMatroid: 5 elements, rank 4>
gap> HasSimpleSimplex(O);
  - tope: [ -1, -1, -1, -1, -1 ]
  - vertices: [ [ 0, 0, 0, -1, -1 ], [ 0, 0, -1, 0, -1 ],
  [ 0, -1, 0, 0, -1 ], [ -1, 0, 0, 0, -1 ] ]
true
```

4.3.3 HasSimpleTriangle (for IsOrientedMatroid)

▷ `HasSimpleTriangle(OM)` (property)

Returns: true or false.

Checks all rank 3 localizations for a simple simplex, i.e. a simple triangle.

Example

4.4 Operations

4.4.1 IsLEquiv (for IsOrientedMatroid,IsOrientedMatroid)

▷ `IsLEquiv(OM1, OM2)` (operation)

Returns: true or false

Determines if the oriented matroids *OM1* and *OM2* have isomorphic geometric lattices.

Example
```
gap> O1:=OrientedMatroid(3,4,[1,1,1,1]);
<OrientedMatroid: 4 elements, rank 3>
gap> O2:=OrientedMatroid([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<OrientedMatroid: 4 elements, rank 3>
```

```
gap> IsLEquiv(O1,O2);
true
gap> O2:=OrientedMatroid([[1,0,0],[0,1,0],[0,0,1],[1,1,0]]);
<OrientedMatroid: 4 elements, rank 3>
gap> IsLEquiv(O1,O2);
false
```

4.4.2 HasSimpleSimplexRk (for IsOrientedMatroid, IsInt)

▷ `HasSimpleSimplexRk(OM, k)` (operation)

Returns: true or false

Returns whether the oriented matroid *OM* has a localiazion of rank k with a simple Simplex. If this is true for $k \geq 3$ then its Salvetti complex can not be $K(\pi, 1)$.

Example

```
gap> O:=OrientedMatroid([[1,0,0,0],[0,1,0,0],[0,0,1,0],[0,0,0,1],[1,1,1,0]]);
<OrientedMatroid: 5 elements, rank 4>
gap> HasSimpleSimplex(O);
false
gap> HasSimpleSimplexRk(O,3);
true
```

4.4.3 OMLocalizationRk (for IsOrientedMatroid, IsList, IsInt)

▷ `OMLocalizationRk(OM, F, k)` (operation)

Returns: An oriented matroid

Returns the rank k localization of *OM* at the flat *F*.

Example

```
gap> O:=OrientedMatroid([[1,0,0,0],[0,1,0,0],[0,0,1,0],[0,0,0,1],[1,1,1,0]]);
<OrientedMatroid: 5 elements, rank 4>
gap> L:=OMGeomLattice(O);
<Geometric lattice: 5 atoms, rank 4>
gap> LkFlats(L)(3);
[ [ 1, 2, 3, 5 ], [ 1, 2, 4 ], [ 1, 3, 4 ], [ 2, 3, 4 ], [ 1, 4, 5 ], [ 2, 4, 5 ], [ 3, 4, 5 ] ]
gap> OMLocalizationRk(O,[1,2,3,5],3);
<OrientedMatroid: 4 elements, rank 3>
```

4.5 Global methods

4.5.1 OMOperation

▷ `OMOperation(sv1, sv2)` (function)

Returns: A list

Performs the oriented matroid operation on two signed vectors `sv1` and `sv2`. For each position `i`, if `sv1[i]` is nonzero, it is used; otherwise `sv2[i]` is used. Returns a new signed vector representing the result of the operation.

Example

```
gap> CVs:=OMCovectors(O)[2];
[ [ 0, 1, 1 ], [ 0, -1, -1 ], [ 1, 0, 1 ],
  [ -1, 0, -1 ], [ -1, 1, 0 ], [ 1, -1, 0 ] ]
gap> sv1 := CVs[3]; sv2 := CVs[5];
[ 1, 0, 1 ]
[ -1, 1, 0 ]
gap> OMOperation(sv1, sv2);
[ 1, 1, 1 ]
gap> OMOperation(sv2, sv1);
[ -1, 1, 1 ]
```

4.5.2 OrderCovec

▷ `OrderCovec(arg)` (function)

Order function for sign vectors.

4.5.3 IsOMEquiv

▷ `IsOMEquiv(OM1, OM2)` (function)

Returns: true or false

Determines, if the oriented matroids *OM1* and *OM2* are isomorphic.

4.5.4 SeparatingSet

▷ `SeparatingSet(arg)` (function)

The separating set of two sign vectors.

Chapter 5

Free arrangements

The following describes functions to analyse freeness properties of arrangements such as *inductive freeness* due to H. Terao [Ter80] and *divisionell freeness* due to T. Abe [Abe16].

5.1 Module of logarithmic vector fields aka derivation modules of arrangements

The functions in this section all use the SINGULAR package to call functions from SINGULAR [DGPS24] for commutative algebra calculations.

5.1.1 DerModule (for IsHyperplaneArrangement)

▷ `DerModule(A[, mult])` (operation)

Returns: Derivation module

Computes the *(multi) derivation module* of the arrangement `A` (with multiplicities given as a list by `mult`).

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
gap> D:=DerModule(A);
<Derivation module
  over PolynomialRing( Rationals, ["x_1", "x_2", "x_3"] )
  with 4 generators>
gap> D:=DerModule(A,[2,1,3,4]);
<Derivation module
  over PolynomialRing( Rationals, ["x_1", "x_2", "x_3"] )
  with 4 generators>
```

5.1.2 DerModGenerators (for IsDerivationModule)

▷ `DerModGenerators(D)` (attribute)

Returns: list

Returns the (minimal) generators of the module `D`.

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
```

```
gap> D:=DerModule(A);
<Derivation module
  over PolynomialRing( Rationals, ["x_1", "x_2", "x_3"] )
  with 4 generators>
gap> DerModGenerators(D);
[ [ x_1, x_2, x_3 ],
  [ 0, -x_2*x_3, x_2*x_3 ],
  [ 0, x_2*x_3, x_1*x_3+x_3^2 ],
  [ 0, x_1*x_2+x_2^2+x_2*x_3, 0 ] ]
```

5.1.3 DerModPRing (for IsDerivationModule)

▷ `DerModPRing(D)` (attribute)

Returns: Polynomial ring

Returns the defining polynomial ring of D.

Example

```
gap> D:=DerModule(AGpql(3,3,2)); DerModPRing(D);
<field in characteristic 0>[x_1,x_2]
```

5.1.4 DerModDegreeSequence (for IsDerivationModule)

▷ `DerModDegreeSequence(D)` (attribute)

Returns: list

Returns the sequence of degrees of (minimal) generators of the graded module D.

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> DerModDegreeSequence(DerModule(A));
[ 1, 2, 2 ]
```

5.1.5 DerModProjDim (for IsDerivationModule)

▷ `DerModProjDim(D)` (attribute)

Returns: A non-negative integer

Computes the projective dimension of the module D.

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
gap> D:=DerModule(A);
<Derivation module
  over PolynomialRing( Rationals, ["x_1", "x_2", "x_3"] )
  with 4 generators>
gap> DerModProjDim(D);
1
```

5.1.6 DerModIsFree (for IsDerivationModule)

▷ `DerModIsFree(D)` (attribute)

Returns: true or false

Determines, if D is a free module over `DerModPRing(D)`.

Example
```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> D:=DerModule(A); DerModIsFree(D);
true
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
gap> D:=DerModule(A); DerModIsFree(D);
false
```

5.1.7 HArrIsFree (for IsHyperplaneArrangement)

▷ `HArrIsFree(A)` (attribute)

Returns: true or false

Determines, if *A* is free, i.e. if the derivation module is a free module over the coordinate ring of the ambient space.

Example
```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> HArrIsFree(A);
true
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
gap> HArrIsFree(A);
false
```

5.2 Freeness properties

5.2.1 IsInductivelyFree (for IsHyperplaneArrangement)

▷ `IsInductivelyFree(A)` (property)

Returns: true or false.

Determines whether the arrangement *A* is *inductively free*.

Example
```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> IsInductivelyFree(A);
true
```

5.2.2 IsDivisionallyFree (for IsHyperplaneArrangement)

▷ `IsDivisionallyFree(A)` (property)

Returns: true or false.

Determines whether the arrangement *A* is *divisionally free*.

Example
```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> IsDivisionallyFree(A);
true
```

```
gap> A:=AGpql(3,3,3);
<HyperplaneArrangement: 9 hyperplanes in 3-space>
gap> IsDivisionallyFree(A);
false
```

Chapter 6

Further properties of arrangements, geometric lattices, and oriented matroids

The following describes functions to analyse further properties of arrangements such as *formality*, *supersolvability*, *simpliciality*, *factoredness*, *inductive factoredness* ...

6.1 Supersolvable arrangements

6.1.1 LIsModularPair (for IsGeomLattice, IsList, IsList)

▷ `LIsModularPair(L, m1, m2)` (operation)

Returns: true or false

Determines if *m1* and *m2* form a modular pair in L.

Example

```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> L:=IntersectionLattice(A);
<Geometric lattice: 6 atoms, rank 3>
gap> LGroundSet(L);
[ [ [ 1 ], [ 2 ], [ 3 ], [ 4 ], [ 5 ], [ 6 ] ],
 [ [ 1, 2 ], [ 1, 3, 6 ], [ 2, 3, 5 ], [ 1, 4, 5 ],
   [ 2, 4, 6 ], [ 3, 4 ], [ 5, 6 ] ],
  [ [ 1, 2, 3, 4, 5, 6 ] ] ]
gap> m1:=[1,2]; m2:= [5,6]; LIsModularPair(L,m1,m2);
false
gap> m1:=[1,2]; m2:= [2,4,6]; LIsModularPair(L,m1,m2);
true
```

6.1.2 LIsModularFlat (for IsGeomLattice, IsList)

▷ `LIsModularFlat(L, m)` (operation)

Returns: true or false

Determines if *m* is a modular flat in L.

Example

```
gap> A:=AGpql(2,2,3); L:=IntersectionLattice(A); LGroundSet(L);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
```

```
<Geometric lattice: 6 atoms, rank 3>
[ [ [ 1 ], [ 2 ], [ 3 ], [ 4 ], [ 5 ], [ 6 ] ],
 [ [ 1, 2 ], [ 1, 3, 6 ], [ 2, 3, 5 ], [ 1, 4, 5 ],
   [ 2, 4, 6 ], [ 3, 4 ], [ 5, 6 ] ],
  [ [ 1, 2, 3, 4, 5, 6 ] ] ]
gap> m1:=[1,2]; m2:= [2,4,6]; LIsModularFlat(L,m1); LIsModularFlat(L,m2);
false
true
```

6.1.3 LModularFlatsRk (for IsGeomLattice, IsInt)

▷ `LModularFlatsRk(L, k)` (operation)

Returns: list

Determines the modular flats in L of rank k.

Example

```
gap> L:=IntersectionLattice(AGpql(2,1,4));
<Geometric lattice: 16 atoms, rank 4>
gap> LModularFlatsRk(L,3);
[ [ 1, 2, 3, 5, 6, 7, 8, 11, 12 ],
  [ 1, 2, 4, 5, 6, 9, 10, 13, 14 ],
  [ 1, 3, 4, 7, 8, 9, 10, 15, 16 ],
  [ 2, 3, 4, 11, 12, 13, 14, 15, 16 ] ]
```

6.1.4 LIsSupersolvable (for IsGeomLattice)

▷ `LIsSupersolvable(L)` (property)

Returns: true or false.

Determines whether the geometric lattice L is *supersolvable*.

Example

```
gap> L:=IntersectionLattice(AGpql(2,1,4));
<Geometric lattice: 16 atoms, rank 4>
gap> LIsSupersolvable(L);
true
gap> L:=IntersectionLattice(AGpql(2,2,4));
<Geometric lattice: 12 atoms, rank 4>
gap> LIsSupersolvable(L);
false
```

6.1.5 HArrIsSupersolvable (for IsHyperplaneArrangement)

▷ `HArrIsSupersolvable(A)` (property)

Returns: true or false.

Determines whether the arrangement A is *supersolvable*.

Example

```
gap> A:=AGpql(2,1,4);
<HyperplaneArrangement: 16 hyperplanes in 4-space>
gap> HArrIsSupersolvable(A);
true
gap> A:=AGpql(2,2,4);
<HyperplaneArrangement: 12 hyperplanes in 4-space>
```

```
gap> HArrIsSupersolvable(A);
false
```

6.1.6 OMIsSupersolvable (for IsOrientedMatroid)

▷ `OMIsSupersolvable(OM)` (property)

Returns: true or false.

Determines whether the oriented matroid `OM` is *supersolvable*.

Example

```
gap> O:=OrientedMatroid(3,4,[1,1,1,1]);
<OrientedMatroid: 4 elements, rank 3>
gap> OMIsSupersolvable(O);
false
gap> C:=List(Combinations(Roots(AGpql(2,2,3)),3),x->pos(Determinant(x)));
[ 1, 1, 1, 1, 0, -1, -1, -1, -1, 0, 1, 0, 1, -1, 0, 1, 1, 1, -1, -1 ]
gap> O:=OrientedMatroid(3,6,C);
<OrientedMatroid: 6 elements, rank 3>
gap> OMIsSupersolvable(O);
true
```

6.2 Formal arrangements

6.2.1 IsFormal (for IsHyperplaneArrangement)

▷ `IsFormal(A)` (property)

Returns: true or false.

Determines whether the arrangement is *formal* in the sense of Falk-Randell [FR87].

Example

```
gap> A1 := Arr([
>     [1,0,0],
>     [0,1,0],
>     [0,0,1],
>     [1,1,1],
>     [2,1,1],
>     [2,3,1],
>     [2,3,4],
>     [3,0,5],
>     [3,4,5]
> ]);
gap>
gap> A2 := Arr([
>     [1,0,0],
>     [0,1,0],
>     [0,0,1],
>     [1,1,1],
>     [2,1,1],
>     [2,3,1],
>     [2,3,4],
>     [1,0,3],
>     [1,2,3]
```

```
> ]);
gap> IsLEquiv(A1,A2);
true
gap> IsFormal(A1);
true
gap> IsFormal(A2);
false
```

6.3 Simplicial arrangements

6.3.1 HArrIsSimplicial (for IsHyperplaneArrangement and IsReal)

▷ `HArrIsSimplicial(A)` (property)

Returns: true or false.

Determines whether the real arrangement *A* is *simplicial*, i.e. if all chambers are simplicial.

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[0,1,1]]); HArrIsSimplicial(A);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
false
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[0,1,1],[1,1,1]]); HArrIsSimplicia
<HyperplaneArrangement: 6 hyperplanes in 3-space>
true
```

6.4 Falk's weight test

The following still needs to be tested further...

6.4.1 OMBoundedCpx (for IsOrientedMatroid, IsInt)

▷ `OMBoundedCpx(OM, g)` (operation)

Returns: list of list of sign vectors.

Constructs the complex of bounded cells of the affine part of the oriented matroid *OM* with respect to the element *g*.

Example

```
```

6.4.2 OMSupportsFalkWeights (for IsOrientedMatroid, IsInt)

▷ `OMSupportsFalkWeights(OM, g)` (operation)

Returns: list or fail

Only for oriented matroids of rank 3. Determines, whether *OM* supports a system of weights as described by M. Falk in [Fal95]. This implies, that the Salvetti complex (see `SalvettiComplex` (4.2.8)) is apsherical. The functionality of the CDDINTERFACE package is used, to determine solotions of a system of linear inequalities.

If the algorithm determines a suitable weight system for *OM*, each list element consists of a weight, and a corner, i.e. a pair of a 2-cell and an adjacent vertex in the bounded complex, given as sign vectors.

Example
```
gap> O:=OrientedMatroid(
>       [[1,1,0],[1,-1,0],[1,0,1],[1,0,-1],[0,1,1],[0,1,-1],[0,0,1]]
> );
<OrientedMatroid: 7 elements, rank 3>
gap> OMIsSupersolvable(O);
false
gap> OMSupportsFalkWeights(O,3);
fail
gap> OMSupportsFalkWeights(O,1);
[ [ 1/2, [ [ 1, 1, 1, 1, 1, 1, 1 ], [ 1, 0, 1, 0, 1, 0, 1 ] ] ],
  [ 0, [ [ 1, 1, 1, 1, 1, 1, 1 ], [ 1, 1, 1, 1, 0, 0, 0 ] ] ],
  [ 1/2, [ [ 1, 1, 1, 1, 1, 1, 1 ], [ 1, 0, 1, 1, 1, 1, 0 ] ] ],
  [ 0, [ [ 1, 1, 1, 1, 1, 1, -1 ], [ 1, 0, 0, 1, 0, 1, -1 ] ] ],
  [ 1/2, [ [ 1, 1, 1, 1, 1, 1, -1 ], [ 1, 1, 1, 1, 0, 0, 0 ] ] ],
  [ 1/2, [ [ 1, 1, 1, 1, 1, 1, -1 ], [ 1, 0, 1, 1, 1, 1, 0 ] ] ],
  [ 0, [ [ 1, -1, 1, 1, 1, 1, 1 ], [ 1, 0, 1, 0, 1, 0, 1 ] ] ],
  [ 1/2, [ [ 1, -1, 1, 1, 1, 1, 1 ], [ 1, -1, 0, 0, 1, 1, 0 ] ] ],
  [ 1/2, [ [ 1, -1, 1, 1, 1, 1, 1 ], [ 1, 0, 1, 1, 1, 1, 0 ] ] ],
  [ 1/2, [ [ 1, -1, 1, 1, 1, 1, -1 ], [ 1, 0, 0, 1, 0, 1, -1 ] ] ],
  [ 0, [ [ 1, -1, 1, 1, 1, 1, -1 ], [ 1, -1, 0, 0, 1, 1, 0 ] ] ],
  [ 1/2, [ [ 1, -1, 1, 1, 1, 1, -1 ], [ 1, 0, 1, 1, 1, 1, 0 ] ] ] ]
```

6.5 Factored arrangements

6.5.1 HArrFactorizations (for IsHyperplaneArrangement)

▷ `HArrFactorizations(A)` (operation)

Returns: A list

Computes a list of all factorizations of A as partitions of $[1..|A|]$.

Example
```
gap> A:=AGpql(2,1,3);
<HyperplaneArrangement: 9 hyperplanes in 3-space>
gap> HArrFactorizations(A);
[ [ [ 1 ], [ 2, 4, 5 ], [ 3, 6, 7, 8, 9 ] ],
  [ [ 1 ], [ 2, 4, 5, 8, 9 ], [ 3, 6, 7 ] ],
  [ [ 2 ], [ 1, 4, 5 ], [ 3, 6, 7, 8, 9 ] ],
  [ [ 2 ], [ 1, 4, 5, 6, 7 ], [ 3, 8, 9 ] ],
  [ [ 3 ], [ 1, 6, 7 ], [ 2, 4, 5, 8, 9 ] ],
  [ [ 3 ], [ 1, 4, 5, 6, 7 ], [ 2, 8, 9 ] ],
  [ [ 4 ], [ 1, 2, 5 ], [ 3, 6, 7, 8, 9 ] ],
  [ [ 5 ], [ 1, 2, 4 ], [ 3, 6, 7, 8, 9 ] ],
  [ [ 6 ], [ 1, 3, 7 ], [ 2, 4, 5, 8, 9 ] ],
  [ [ 7 ], [ 1, 3, 6 ], [ 2, 4, 5, 8, 9 ] ],
  [ [ 8 ], [ 1, 4, 5, 6, 7 ], [ 2, 3, 9 ] ],
  [ [ 9 ], [ 1, 4, 5, 6, 7 ], [ 2, 3, 8 ] ] ]
```

6.5.2 HArrIsFactored (for IsHyperplaneArrangement)

▷ `HArrIsFactored(A)` (attribute)

Returns: A list or fail

Determines if A is factored and if so returns some factorization.

Example
```
gap> A:=AGpql(2,1,3);
<HyperplaneArrangement: 9 hyperplanes in 3-space>
gap> HArrIsFactored(A);
[ [ 1 ], [ 2, 4, 5 ], [ 3, 6, 7, 8, 9 ] ]
gap> A:=AGpql(2,2,4);
<HyperplaneArrangement: 12 hyperplanes in 4-space>
gap> HArrIsFactored(A);
fail
```

6.5.3 HArrInductiveFactorizations (for IsHyperplaneArrangement)

▷ `HArrInductiveFactorizations(A)` (operation)

Returns: A list

Computes a list of all inductive factorizations of A as partitions of $[1..|A|]$.

Example
```
gap> A:=AGpql(2,1,3);
<HyperplaneArrangement: 9 hyperplanes in 3-space>
gap> HArrInductiveFactorizations(A);
[ [ [ 1 ], [ 2, 4, 5 ], [ 3, 6, 7, 8, 9 ] ],
  [ [ 1 ], [ 2, 4, 5, 8, 9 ], [ 3, 6, 7 ] ],
  [ [ 2 ], [ 1, 4, 5 ], [ 3, 6, 7, 8, 9 ] ],
  [ [ 2 ], [ 1, 4, 5, 6, 7 ], [ 3, 8, 9 ] ],
  [ [ 3 ], [ 1, 6, 7 ], [ 2, 4, 5, 8, 9 ] ],
  [ [ 3 ], [ 1, 4, 5, 6, 7 ], [ 2, 8, 9 ] ],
  [ [ 4 ], [ 1, 2, 5 ], [ 3, 6, 7, 8, 9 ] ],
  [ [ 5 ], [ 1, 2, 4 ], [ 3, 6, 7, 8, 9 ] ],
  [ [ 6 ], [ 1, 3, 7 ], [ 2, 4, 5, 8, 9 ] ],
  [ [ 7 ], [ 1, 3, 6 ], [ 2, 4, 5, 8, 9 ] ],
  [ [ 8 ], [ 1, 4, 5, 6, 7 ], [ 2, 3, 9 ] ],
  [ [ 9 ], [ 1, 4, 5, 6, 7 ], [ 2, 3, 8 ] ] ]
```

6.5.4 HArrIsInductivelyFactored (for IsHyperplaneArrangement)

▷ `HArrIsInductivelyFactored(A)` (attribute)

Returns: A list or fail

Determines if A is inductively factored and if so returns some inductive factorization.

Example
```
gap> A:=AGpql(2,1,4);
<HyperplaneArrangement: 16 hyperplanes in 4-space>
gap> HArrIsInductivelyFactored(A);
[ [ [ 1 ], [ 2, 5, 6 ], [ 3, 7, 8, 11, 12 ], [ 4, 9, 10, 13, 14, 15, 16 ] ], true ]
gap> A:=AGpql(2,2,4);
<HyperplaneArrangement: 12 hyperplanes in 4-space>
gap> HArrIsInductivelyFactored(A);
false
```

6.5.5 HArrIsInductivelyFactored (for IsHyperplaneArrangement,IsString)

▷ `HArrIsInductivelyFactored(arg1, arg2)` (operation)

Chapter 7

Milnor fibers

The Milnor fiber is the typical fiber $f^{-1}(1)$ of the Milnor fibration

$$f : \mathbb{C}^\ell \setminus \cup_{H\in\mathscr{A}} H \to \mathbb{C}^\times, z \mapsto \prod_{H\in\mathscr{A}} \alpha_H(z)$$

of the hyperplane arrangement $\mathscr{A}$, where α_H are defining linear forms.

The following describes functions to construct a regular cell complex having the homotopy type of the Milnor fiber from [MY25] and thus enables computation of homotopy invaraints using e.g. the HAP package [Ell26].

7.1 Complexes

7.1.1 MilnorFiberComplex (for IsOrientedMatroid)

▷ `MilnorFiberComplex(OM)` (attribute)

Returns: FacePoset

Computes the face poset of the regular cell complex of the combinatorial Milnor fiber of the oriented matroid *OM*.

Example

```
gap> O:=OrientedMatroid([[1,0],[0,1],[1,1]]);
<OrientedMatroid: 3 elements, rank 2>
gap> MCpx:=MilnorFiberComplex(O);
<FacePoset of dimension 1 with f-vector [ 3, 6 ]>
```

7.1.2 MilnorFiberComplex (for IsHyperplaneArrangement)

▷ `MilnorFiberComplex(A)` (attribute)

Returns: FacePoset

Computes the face poset of the regular cell complex having the homotopy type of Milnor fiber of the arrangement *A*.

Example

```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> MCpx:=MilnorFiberComplex(A);
<FacePoset of dimension 2 with f-vector [ 12, 60, 60 ]>
```

7.1.3 FPtoCWCpx

▷ `FPtoCWCpx(FP)` (function)

Returns: HAP CW Complex

Converts a HYPARR `FacePoset` into a HAP `RegularCWComplex` for further computations of topological invariants.

Example

```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> MCpx:=MilnorFiberComplex(A);
<FacePoset of dimension 2 with f-vector [ 12, 60, 60 ]>
gap> MCW := FPtoCWCpx(MCpx);
Regular CW-complex of dimension 2
gap> Homology(MCW,0);
[ 0 ]
gap> Homology(MCW,1);
[ 0, 0, 0, 0, 0, 0, 0 ]
```

Chapter 8

Tope posets

8.1 Operation

8.1.1 TopePoset (for IsOrientedMatroid, IsList)

▷ `TopePoset(OM, BTope)` (operation)

Returns: A TopePoset object

Constructs the tope poset of the oriented matroid *OM* with respect to the base tope *BTope* (given as sign vector).

Example

```
gap> O:=OM([[1,0],[0,1]]); Ts:=OMCovectors(O)[1]; T:=Ts[1];
<OrientedMatroid: 2 elements, rank 2>
[ [ -1, -1 ], [ 1, 1 ], [ -1, 1 ], [ 1, -1 ] ]
[ -1, -1 ]
gap> TP:=TopePoset(O,T);
<TopePoset with 3 topes>
```

8.2 Attributes

8.2.1 TPGroundSet (for IsTopePoset)

▷ `TPGroundSet(TP)` (attribute)

Returns: A list of lists.

The ground set of the tope poset *TP*. It is a list of list, recording the separating elements of the given tope from the base tope.

Example

```
gap> TPGroundSet(TP);
[ [ [  ] ], [ [ 1 ], [ 2 ] ], [ [ 1, 2 ] ] ]
```

8.2.2 TPBaseTope (for IsTopePoset)

▷ `TPBaseTope(TP)` (attribute)

Returns: A sign vector.

Returns the bast tope of the tope poset *TP*.

Example
```
gap> TPBaseTope(TP);
[ -1, -1 ]
```

8.2.3 TPRankPoly (for IsTopePoset)

▷ TPRankPoly(*TP*) (attribute)

Returns: A polynomial.

Returns the rank generating polynomial of the tope poset *TP*.

Example
```
gap> TPRankPoly(TP);
t^2+2*t+1
```

Chapter 9

Visualization of real 3-arrangements

9.1 Global methods

9.1.1 LaTeXDrawProjPicture

▷ `LaTeXDrawProjPicture(A[, DrawOptions])` (function)

Returns: A string.

Generates LaTeX tikz-code for a nice projective picture of the real 3-arrangement. If x, y, z are the coordinates, by default, this is the 2-dim affine arrangement obtained by intersecting `A` with the plane $z = 1$.

In the optional parameters `DrawOptions`, a record, the following parameters can be chosen

- `scale`, a positive rational scaling factor,
- `isecps` intersection points are drawn, `true` or `false`,
- `Hind` labels for the hyperplanes are added, `true` or `false`,
- `deconeH` a vector giving the normal of the plane (=1) with which to intersect `A`,
- `MarkHs` a list of indices of hyperplanes of `A`, these are drawn in another color.

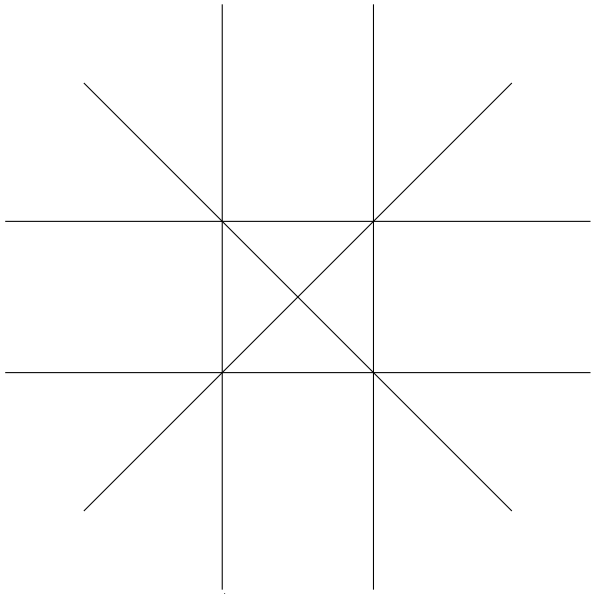
The example below will look as follows (only in pdf or pic).

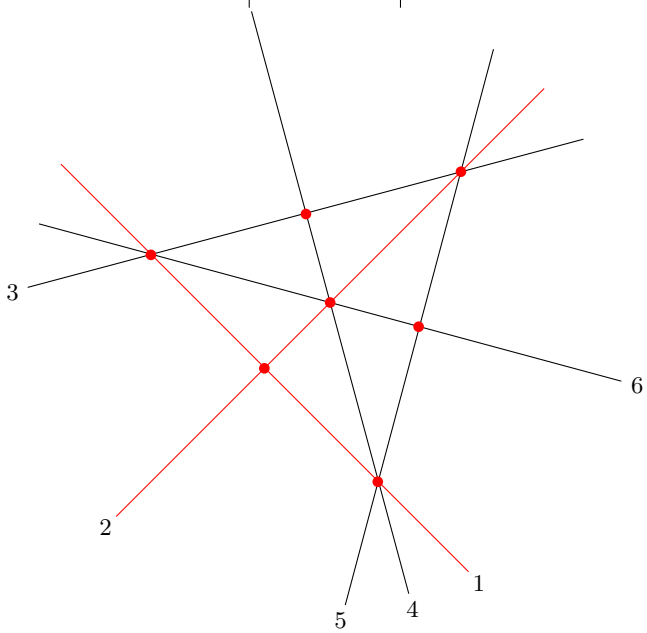
Example

```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> Print(LaTeXDrawProjPicture(A));
\begin{tikzpicture}[scale=1.0]
\draw (-2.83,2.83) -- (2.83,-2.83);  % H_1
\draw (2.83,2.83) -- (-2.83,-2.83);  % H_2
\draw (-1.,3.87) -- (-1.,-3.87);  % H_3
\draw (1.,3.87) -- (1.,-3.87);  % H_4
\draw (3.87,-1.) -- (-3.87,-1.);  % H_5
\draw (3.87,1.) -- (-3.87,1.);  % H_6
\end{tikzpicture}
gap> DrawOpts:=rec(scale:=1/2,isecps:=true,Hind:=true,
>       deconeH:=[1,1,1],MarkHs:=[1,2]);;
gap> Print(LaTeXDrawProjPicture(AGpql(2,2,3),DrawOpts));
\begin{tikzpicture}[scale=1.0]
```

```
\draw[color=red] (-3.56,1.83) -- (1.83,-3.56);  % H_1
\node at (1.97,-3.71) {\small $1$};
\draw[color=red] (2.83,2.83) -- (-2.83,-2.83);  % H_2
\node at (-2.97,-2.97) {\small $2$};
\draw (3.35,2.16) -- (-4.,0.20);  % H_3
\node at (-4.20,0.14) {\small $3$};
\draw (-1.04,3.85) -- (1.04,-3.85);  % H_4
\node at (1.09,-4.06) {\small $4$};
\draw (2.16,3.35) -- (0.20,-4.);  % H_5
\node at (0.14,-4.20) {\small $5$};
\draw (-3.85,1.04) -- (3.85,-1.04);  % H_6
\node at (4.06,-1.09) {\small $6$};

\fill[red] (-0.87,-0.87) circle[radius=2pt];  % P[ 1, 2 ]
\fill[red] (-2.37,0.63) circle[radius=2pt];  % P[ 1, 3, 6 ]
\fill[red] (1.73,1.73) circle[radius=2pt];  % P[ 2, 3, 5 ]
\fill[red] (0.63,-2.37) circle[radius=2pt];  % P[ 1, 4, 5 ]
\fill[red] (0.0,0.0) circle[radius=2pt];  % P[ 2, 4, 6 ]
\fill[red] (-0.32,1.17) circle[radius=2pt];  % P[ 3, 4 ]
\fill[red] (1.17,-0.32) circle[radius=2pt];  % P[ 5, 6 ]
\end{tikzpicture}
```





Projective pictures produced by `LaTeXDrawProjPicture`.

9.1.2 LaTeXDrawSpherePicture

▷ `LaTeXDrawSpherePicture(A)` (function)

Returns: A string.

Generates LaTeX tikz-code for the intersection of a real 3-arrangement with the unit sphere. To compile the LaTeX-code the .sty-file "graphonsphere.sty" (from /doc/LaTeX_Examples) needs to be in the same folder and added via "\usepackage{graphonsphere}".

The example below will look as follows (only in pdf or pic).

Example

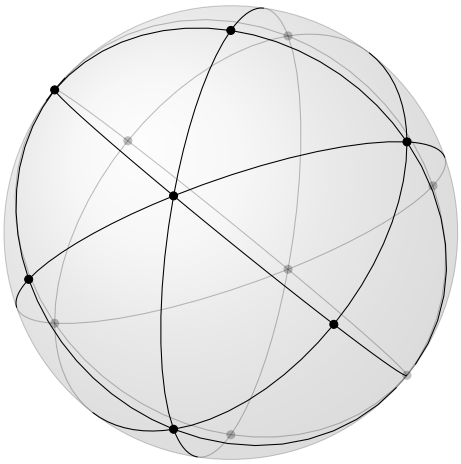
```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
```

```
gap> Print(LaTeXDrawSpherePicture(A));
\tdplotsetmaincoords{63}{63}
\begin{tikzpicture}[scale=3,tdplot_main_coords]

\draw[ball color = gray!40, opacity = 0.2] (0,0,0) circle (1cm);

\def\OMcolor{black}
{\color{\OMcolor}
% the vertices of the OM-cpx
\foreach \i/\vx/\vy/\vz in {
1/0./0./1.,
2/-0.57/0.57/0.57,
3/-0.57/-0.57/0.57,
4/0.57/-0.57/0.57,
5/0.57/0.57/0.57,
6/0./1./0.,
7/1./0./0.,
8/0./0./-1.,
9/0.57/-0.57/-0.57,
10/0.57/0.57/-0.57,
11/-0.57/0.57/-0.57,
12/-0.57/-0.57/-0.57,
13/0./-1./0.,
14/-1./0./0.
}{
\node (\i) at (\vx,\vy,\vz) {};
\POnSfb{\vx}{\vy}{\vz}{}
}
%
% the edges of the OM-cpx
\foreach \ax/\ay/\az/\bx/\by/\bz in {
0./0./1./  -0.57/0.57/0.57, %[ 1, 2 ]
0./0./1./  -0.57/-0.57/0.57, %[ 1, 3 ]
0./0./1./  0.57/-0.57/0.57, %[ 1, 4 ]
0./0./1./  0.57/0.57/0.57, %[ 1, 5 ]
-0.57/0.57/0.57/  -0.57/-0.57/0.57, %[ 2, 3 ]
-0.57/0.57/0.57/  0.57/0.57/0.57, %[ 2, 5 ]
-0.57/0.57/0.57/  0./1./0., %[ 2, 6 ]
-0.57/0.57/0.57/  -0.57/0.57/-0.57, %[ 2, 11 ]
-0.57/0.57/0.57/  -1./0./0., %[ 2, 14 ]
-0.57/-0.57/0.57/  0.57/-0.57/0.57, %[ 3, 4 ]
-0.57/-0.57/0.57/  -0.57/-0.57/-0.57, %[ 3, 12 ]
-0.57/-0.57/0.57/  0./-1./0., %[ 3, 13 ]
-0.57/-0.57/0.57/  -1./0./0., %[ 3, 14 ]
0.57/-0.57/0.57/  0.57/0.57/0.57, %[ 4, 5 ]
0.57/-0.57/0.57/  1./0./0., %[ 4, 7 ]
0.57/-0.57/0.57/  0.57/-0.57/-0.57, %[ 4, 9 ]
0.57/-0.57/0.57/  0./-1./0., %[ 4, 13 ]
0.57/0.57/0.57/  0./1./0., %[ 5, 6 ]
0.57/0.57/0.57/  1./0./0., %[ 5, 7 ]
0.57/0.57/0.57/  0.57/0.57/-0.57, %[ 5, 10 ]
0./1./0./  0.57/0.57/-0.57, %[ 6, 10 ]
```

```
0./1./0./  -0.57/0.57/-0.57, %[ 6, 11 ]
1./0./0./  0.57/-0.57/-0.57, %[ 7, 9 ]
1./0./0./  0.57/0.57/-0.57, %[ 7, 10 ]
0./0./-1./  0.57/-0.57/-0.57, %[ 8, 9 ]
0./0./-1./  0.57/0.57/-0.57, %[ 8, 10 ]
0./0./-1./  -0.57/0.57/-0.57, %[ 8, 11 ]
0./0./-1./  -0.57/-0.57/-0.57, %[ 8, 12 ]
0.57/-0.57/-0.57/  0.57/0.57/-0.57, %[ 9, 10 ]
0.57/-0.57/-0.57/  -0.57/-0.57/-0.57, %[ 9, 12 ]
0.57/-0.57/-0.57/  0./-1./0., %[ 9, 13 ]
0.57/0.57/-0.57/  -0.57/0.57/-0.57, %[ 10, 11 ]
-0.57/0.57/-0.57/  -0.57/-0.57/-0.57, %[ 11, 12 ]
-0.57/0.57/-0.57/  -1./0./0., %[ 11, 14 ]
-0.57/-0.57/-0.57/  0./-1./0., %[ 12, 13 ]
-0.57/-0.57/-0.57/  -1./0./0. %[ 12, 14 ]
}{
\GCArcABfb{\ax}{\ay}{\az}{\bx}{\by}{\bz}{color=\OMcolor}
}
}
\end{tikzpicture}
```



Sphere picture produced by `LaTeXDrawSpherePicture`.

9.1.3 LaTeXDrawTopeGraph

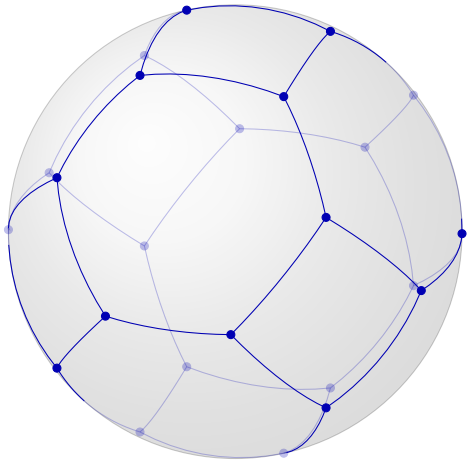
▷ `LaTeXDrawTopeGraph(A)` (function)

Returns: A string.

Generates LaTeX tikz-code for the tope graph of a real 3-arrangement on the unit sphere. To compile the LaTeX-code the .sty-file "graphonsphere.sty" (from /doc/LaTeX_Examples) needs to be in the same folder and added via "\usepackage{graphonsphere}".

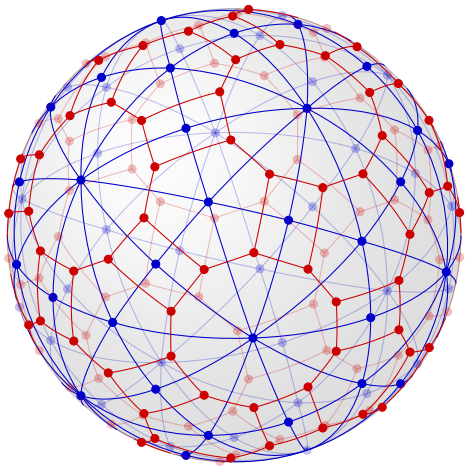
The example below will look as follows (only in pdf or pic).

Example



Tope graph picture produced by `LaTeXDrawTopeGraph`.

Both function `LaTexDrawSpherePicture` and `LaTeXDrawTopeGraph` combined can be used to draw nice pictures of oriented matroid complexes and their duals. See e.g. (only in pdf or pic)



Oriented matroid complex and its dual of $\mathcal{A}(H_3)$.

Chapter 10

Realization spaces of geometric lattices

The computation of the realization space uses the SINGULAR package to call functions from SINGULAR [DGPS24].

10.1 Construction

10.1.1 LRealizationSpace (for IsGeomLattice, IsInt)

▷ `LRealizationSpace(L, char[, GenSetL])` (operation)

Returns: RealizationSpaceOfGeometricLattice

Computes the realization space of the geometric lattice *L* in characteristic *char*. Optionally, a generating set of *L* (as a subset of `LAtoms(L)`) can be given. Otherwise, the algorithm tries to find a small one.

Example

```
gap> L:=IntersectionLattice(AGpql(3,3,3));
<Geometric lattice: 9 atoms, rank 3>
gap> RS:=LRealizationSpace(L,0);
<RealizationSpaceOfGeomLattice: in characteristic 0, non-empty: true>
gap> S:=LGenSet(L);
[ 2, 3, 4, 6, 8 ]
gap> RS:=LRealizationSpace(L,0,S);
<RealizationSpaceOfGeomLattice: in characteristic 0, non-empty: true
```

10.2 Attributes

10.2.1 LGenSet (for IsGeomLattice)

▷ `LGenSet(L)` (attribute)

Returns: a set

Computes some generating set of *L*.

Example

```
gap> L:=IntersectionLattice(AGpql(3,3,3));
<Geometric lattice: 9 atoms, rank 3>
gap> LGenSet(L);
[ 2, 3, 4, 6, 8 ]
```

10.2.2 RSLattice (for IsRealizationSpaceOfGeomLattice)

▷ RSLattice(*RS*) (attribute)

Returns: a geometric lattice

Returns the geometric lattice of the realization space *RS*.

Example

```
gap> RSLattice(RS);
<Geometric lattice: 9 atoms, rank 3>
```

10.2.3 RSCharacteristic (for IsRealizationSpaceOfGeomLattice)

▷ RSCharacteristic(*RS*) (attribute)

Returns: 0 or a prime number

Returns the characteristic of the reliation space *RS*.

Example

```
gap> RSCharacteristic(RS);
0
```

10.2.4 RSDefField (for IsRealizationSpaceOfGeomLattice)

▷ RSDefField(*RS*) (attribute)

Returns: a field

Returns the field over which the realization space *RS* is defined.

Example

```
gap> RSDefField(RS);
Rationals
```

10.2.5 RSCoeffMat (for IsRealizationSpaceOfGeomLattice)

▷ RSCoeffMat(*RS*) (attribute)

Returns: a matrix

Returns the coefficient matrix of *RS* with entries in RSPRing(RS).

Example

```
gap> RSCoeffMat(RS);
[ [ a1-1, a1, 0 ], [ 1, 0, 0 ], [ 0, 1, 0 ], [ 0, 0, 1 ],
  [ 1, 1, a1 ], [ 1, 1, 1 ], [ 0, 1, -a1^2+a1 ],
  [ 1, 0, a1 ], [ -a1+1, -a1, -a1^2 ] ]
```

10.2.6 RSDimension (for IsRealizationSpaceOfGeomLattice)

▷ RSDimension(*RS*) (attribute)

Returns: a non-negative integer

Returns the dimension of *RS*.

Example

```
gap> RSDimension(RS);
0
```

10.2.7 RSPRing (for IsRealizationSpaceOfGeomLattice)

▷ RSPRing(*RS*) (attribute)

Returns: a polynomial ring

The polynomial ring over which the representing coefficient matrix RSCoeffMat(RS) of *RS* is defined.

Example
```
gap> RSPRing(RS);
Rationals[a1]
```

10.2.8 RSIdealMinors (for IsRealizationSpaceOfGeomLattice)

▷ RSIdealMinors(*RS*) (attribute)

Returns: an ideal

Returns the ideal of equations which have to satisfied due to the dependcy of atoms in RSLattice(RS) over RSDefField(RS).

Example
```
gap> I:=RSIdealMinors(RS); GeneratorsOfIdeal(I);
<two-sided ideal in Rationals[a1], (1 generator)>
[ a1^2-a1+1 ]
```

10.2.9 RSNonMinors (for IsRealizationSpaceOfGeomLattice)

▷ RSNonMinors(*RS*) (attribute)

Returns: a list of polynomials

Gives a list of polynomials which should not vanish due to independent subsets of atoms in RSLattice(RS) over RSDefField(RS).

Example
```
gap> RSNonMinors(RS);
[ a1^3-a1^2, a1^3, -a1^3+2*a1^2-a1, -2*a1^2+2*a1-1 ]
```

10.3 Properties

10.3.1 RSIsNonEmpty (for IsRealizationSpaceOfGeomLattice)

▷ RSIsNonEmpty(*RS*) (property)

Returns: true or false

10.4 Further (auxillary) Operations

10.4.1 LSubsetGeneratedByS (for IsGeomLattice,IsList)

▷ LSubsetGeneratedByS(*L*, *S*) (operation)

10.4.2 LIsGenSet (for IsGeomLattice,IsList)

▷ LIsGenSet(*L*, *S*) (operation)

References

[Abe16] T. Abe. Divisionally free arrangements of hyperplanes. *Invent. Math.*, 204(1):317–346, 2016. 22

[BLVS+99] Anders Björner, Michel Las Vergnas, Bernd Sturmfels, Neil White, and Günter M. Ziegler. *Oriented matroids*, volume 46 of *Encyclopedia of Mathematics and its Applications*. Cambridge University Press, Cambridge, second edition, 1999. 4, 14

[CK22] M. Cuntz and L. Kühne. On arrangements of hyperplanes from connected subgraphs, 2022. 13

[DGPS24] Wolfram Decker, Gert-Martin Greuel, Gerhard Pfister, and Hans Schönemann. SINGULAR 4-4-0 — A computer algebra system for polynomial computations. http://www.singular.uni-kl.de, 2024. 22, 43

[Ell26] G. Ellis. HAP, Homological Algebra Programming, Version 1.75. https://gap-packages.github.io/hap, Mar 2026. GAP package. 33

[Fal95] M. Falk. $K(\pi,1)$ arrangements. *Topology*, 34(1):141–154, 1995. 29

[FR87] M. Falk and R. Randell. On the homotopy theory of arrangements. In *Complex analytic singularities*, volume 8 of *Adv. Stud. Pure Math.*, pages 101–124. North-Holland, Amsterdam, 1987. 28

[MY25] P. Mücksch and M. Yoshinaga. Milnor fibrations and oriented matroids. *ArXiv e-prints*, page arXiv:2508.15331, 2025. 33

[OT92] P. Orlik and H. Terao. *Arrangements of Hyperplanes*. Grundlehren der mathematischen Wissenschaften. Springer, 1992. 4

[Sal87] M. Salvetti. Topology of the complement of real hyperplanes in $\mathbf{C}^N$. *Invent. Math.*, 88(3):603–618, 1987. 17

[Ter80] H. Terao. Arrangements of hyperplanes and their freeness I. *J. Fac. Sci. Univ. Tokyo*, 27:293–320, 1980. 22

Index

AGpql, 12

CharPoly
for IsHyperplaneArrangement, 6
ConnectedSubgraphArr
for IsList, 13

DerModDegreeSequence
for IsDerivationModule, 23
DerModGenerators
for IsDerivationModule, 22
DerModIsFree
for IsDerivationModule, 23
DerModPRing
for IsDerivationModule, 23
DerModProjDim
for IsDerivationModule, 23
DerModule
for IsHyperplaneArrangement, 22
Dimension
for IsHyperplaneArrangement, 5

Essentialization
for IsHyperplaneArrangement, 11
ExpArr
for IsHyperplaneArrangement, 7

FPGroundSet
for IsFacePoset, 17
FPOrder
for IsFacePoset, 18
FPtoCWCpx, 34

HArrDefField
for IsHyperplaneArrangement, 6
HArrFactorizations
for IsHyperplaneArrangement, 30
HArrInductiveFactorizations
for IsHyperplaneArrangement, 31
HArrIsFactored
for IsHyperplaneArrangement, 30
HArrIsFree
for IsHyperplaneArrangement, 24
HArrIsGeneric
for IsHyperplaneArrangement, 8
HArrIsInductivelyFactored
for IsHyperplaneArrangement, 31
for IsHyperplaneArrangement,IsString, 32
HArrIsIrreducible
for IsHyperplaneArrangement, 7
HArrIsSimplicial
for IsHyperplaneArrangement and IsReal, 29
HArrIsSupersolvable
for IsHyperplaneArrangement, 27
HArrRestriction
for IsHyperplaneArrangement, IsInt, 11
for IsHyperplaneArrangement, IsList, 11
HasSimpleSimplex
for IsOrientedMatroid, 19
HasSimpleSimplexRk
for IsOrientedMatroid, IsInt, 20
HasSimpleTriangle
for IsOrientedMatroid, 19
HyperplaneArrangement
for IsList, 5

IntersectionLattice
for IsHyperplaneArrangement, 6
IsDivisionallyFree
for IsHyperplaneArrangement, 24
IsFormal
for IsHyperplaneArrangement, 28
IsInductivelyFree
for IsHyperplaneArrangement, 24
IsLEquiv
for IsHyperplaneArrangement,IsHyperplaneArrangement, 10
for IsOrientedMatroid,IsOrientedMatroid, 19

IsOMEquiv, 21
IsReal
for IsHyperplaneArrangement, 7

LaTeXDrawProjPicture, 37
LaTeXDrawSpherePicture, 39
LaTeXDrawTopeGraph, 41
LAtoms
for IsGeomLattice, 8
LAutGroup
for IsGeomLattice, 9
LBases
for IsGeomLattice, 9
LCharPoly
for IsGeomLattice, 9
LGenSet
for IsGeomLattice, 43
LGraph
for IsGeomLattice, 9
LGroundSet
for IsGeomLattice, 8
LIsBoolean
for IsGeomLattice, 9
LIsGeneric
for IsGeomLattice, 10
LIsGenSet
for IsGeomLattice,IsList, 45
LIsIrreducible
for IsGeomLattice, 9
LIsIsomorphic
for IsGeomLattice,IsGeomLattice, 10
LIsModularFlat
for IsGeomLattice, IsList, 26
LIsModularPair
for IsGeomLattice, IsList, IsList, 26
LIsSupersolvable
for IsGeomLattice, 27
LkFlats
for IsGeomLattice, 8
LModularFlatsRk
for IsGeomLattice, IsInt, 27
LMoebius
for IsGeomLattice, 9
LRank
for IsGeomLattice, 8
LRankFunction
for IsGeomLattice, 8
LRealizationSpace
for IsGeomLattice, IsInt, 43
LSubsetGeneratedByS
for IsGeomLattice,IsList, 45

MilnorFiberComplex
for IsHyperplaneArrangement, 33
for IsOrientedMatroid, 33
MSetInvL
for IsHyperplaneArrangement, 7

OMBoundedCpx
for IsOrientedMatroid, IsInt, 29
OMChirotope
for IsOrientedMatroid, 18
OMCocircuits
for IsOrientedMatroid, 16
OMCovectors
for IsOrientedMatroid, 16
OMGeomLattice
for IsOrientedMatroid, 15
OMGroundSet
for IsOrientedMatroid, 15
OMIsLinear
for IsOrientedMatroid, 18
OMIsSupersolvable
for IsOrientedMatroid, 28
OMLForms
for IsOrientedMatroid, 15
OMLocalizationRk
for IsOrientedMatroid, IsList, IsInt, 20
OMOperation, 20
OMRank
for IsOrientedMatroid, 14
OMRankFunction
for IsOrientedMatroid, 18
OMSupportsFalkWeights
for IsOrientedMatroid, IsInt, 29
OrderCovec, 21
OrientedMatroid
for IsList, 14

Roots
for IsHyperplaneArrangement, 6
RSCharacteristic
for IsRealizationSpaceOfGeomLattice, 44
RSCoeffMat

for IsRealizationSpaceOfGeomLattice, 44
RSDefField
for IsRealizationSpaceOfGeomLattice, 44
RSDimension
for IsRealizationSpaceOfGeomLattice, 44
RSIdealMinors
for IsRealizationSpaceOfGeomLattice, 45
RSIsNonEmpty
for IsRealizationSpaceOfGeomLattice, 45
RSLattice
for IsRealizationSpaceOfGeomLattice, 44
RSNonMinors
for IsRealizationSpaceOfGeomLattice, 45
RSPRing
for IsRealizationSpaceOfGeomLattice, 45

SalvettiComplex
for IsHyperplaneArrangement, 17
for IsOrientedMatroid, 17
SeparatingSet, 21
SsS3, 12

TopeGraph
for IsOrientedMatroid, 16
TopePoset
for IsOrientedMatroid, IsList, 35
TPBaseTope
for IsTopePoset, 35
TPGroundSet
for IsTopePoset, 35
TPRankPoly
for IsTopePoset, 36